

EVCharging Network Release 2 - Sistemas Distribuidos (25/26)

Práctica Sockets, Streaming de Eventos y Colas

Pablo Lecocq Hernandis - 51775490Y
David Quevedo Mora - 51257546E

07 / 01 / 2026

1. Resumen.....	3
2. Objetivos y alcance.....	3
3. Arquitectura General.....	4
3.1 Vista global.....	4
3.2 Canales de comunicación.....	5
3.3 Soporte web y persistencia.....	5
3.4 Justificación de la separación Monitor / Engine.....	5
4. Componentes y Protocolos.....	6
4.1. EV_Central (orquestración, API/WS, persistencia y sockets).....	6
4.2 EV_CP_M (Monitor del punto).....	8
4.3 EV_CP_E (Engine del punto).....	10
4.4 EV_Driver (Cliente solicitante).....	13
4.5 EV_W (Servicio Weather).....	15
4.6 EV_Registry.....	15
4.7 Protocolos y tópicos (resumen).....	15
5. Flujos principales.....	17
5.1 Registro del CP y heartbeats (Registry ↔ Monitor ↔ Central // Monitor ↔ Engine). 17	
5.2 Solicitud del Driver y autorización (Driver → Central → Engine) + telemetrías y ticket final.....	19
5.3 Comandos de Central (STOP / RESUME) a uno o a todos los CP.....	21
5.4 Cierre de sesión y persistencia (Central).....	22
5.5 Detección de desconexión (timeout de heartbeat).....	24
5.6 Control automático por clima (EV_W → Central → CPs).....	25
5.7 Cambio de ubicación en caliente (Monitor → Central).....	26
6. Panel de monitorización.....	27
7. Mecanismos de cifrado.....	28
7.1 Cifrado del canal entre EV_CP_M y EV_Registry (HTTPS).....	28
7.2 Protección segura de credenciales de los CPs.....	28
Almacenamiento seguro en Registry y Central.....	28
7.3 Autenticación segura entre EV_CP_M y EV_Central.....	29
7.4 Cifrado de mensajes entre CPs (Engine/Monitor) y EV_Central.....	29
Algoritmo utilizado.....	29
Contenido del AAD.....	29
7.5 Protección frente a ataques de repetición (Replay Attacks).....	29
7.6 Revocación de claves de cifrado.....	30
8. Modelo de datos.....	30
8.1 Registro de auditoría.....	32
9. Guía de despliegue.....	32
Instalación y preparación previa del sistema.....	32
Comenzamos el despliegue.....	35
Ejemplo del despliegue.....	39
10. Aplicaciones extra utilizadas.....	46

1. Resumen

Esta práctica implementa una simulación de una red de puntos de recarga de vehículos eléctricos (EVCharging) con una central de monitorización (EV_Central), aplicaciones de punto de recarga separadas en Monitor (EV_CP_M) y Engine (EV_CP_E), y una aplicación de conductor (EV_Driver). Se integran sockets TCP para autenticación/latidos y Kafka para streaming de eventos (autorizaciones, telemetrías, comandos). La central expone una API + panel web con WebSocket para visualizar estados en tiempo real y persiste la información en SQLite.

En esta Release 2 se añade un servicio **EV_W** que integra **OpenWeather** para monitorizar temperatura por localización y aplicar reglas automáticas: si la temperatura de una ubicación baja de 0°C, los CPs asociados se detienen (PARADO) o quedan marcados como “pendientes” si estaban suministrando; al recuperarse la temperatura, se reanudan.

Además, se implementa un **registro de auditoría estructurado** para todos los eventos relevantes (auth, comandos, cambios de estado, weather, revocaciones, etc.) y el **cambio de ubicación en caliente** desde el monitor sin reiniciar procesos.

2. Objetivos y alcance

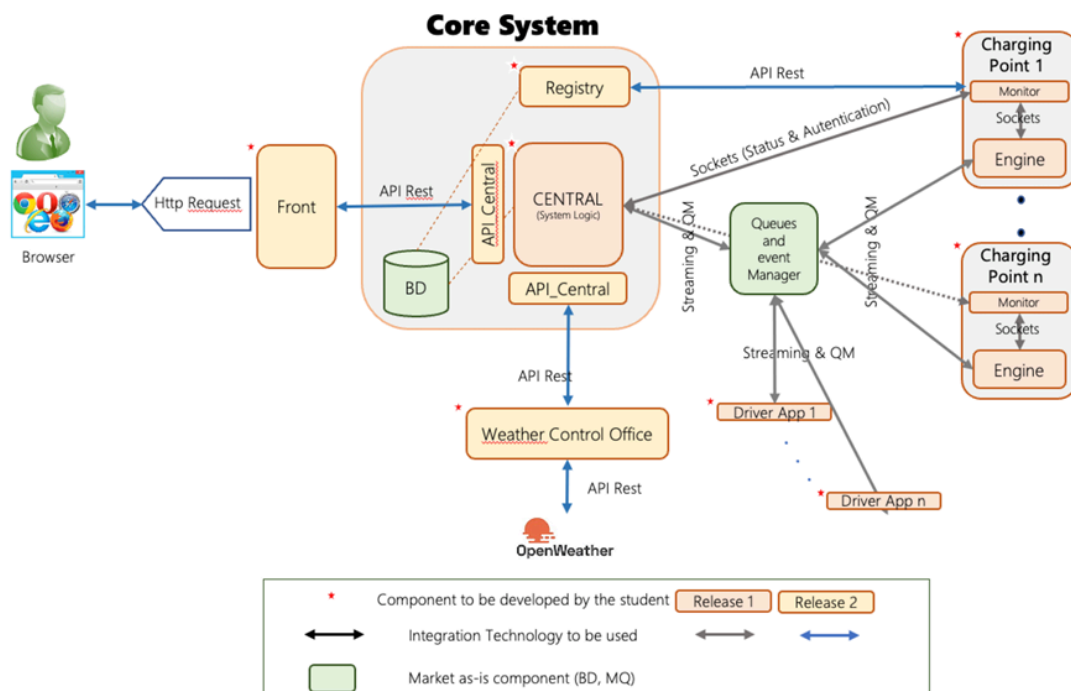
Objetivo: Diseñar y desplegar un sistema distribuido “EVCharging” que gestione puntos de recarga (CP), permita a conductores solicitar servicio y muestre en tiempo real el estado de la red desde una central.

Alcance:

- **CENTRAL:** gobierna estados, autoriza servicios, muestra panel, registra histórico.
- **CP:** dividido en **Monitor (EV_CP_M)** para registro/heartbeat y **Engine (EV_CP_E)** para recibir órdenes de CENTRAL y emitir telemetrías.
- **DRIVER (EV_Driver):** solicita servicios uno a uno o leyendo un fichero de CPs, mostrando avances y ticket final.

La práctica integra **sockets** y **Kafka** conforme a la especificación entregada.

3.Arquitectura General



3.1 Vista global

La solución se ha dividido en **cuatro procesos independientes** que se comunican mediante **sockets TCP, Kafka y HTTP/WebSocket** y se apoya en un proveedor externo (**OpenWeather**) para convertir nombres de ciudades en coordenadas (**geocoding**) y obtener la temperatura.

- **EV_Central:** orquesta la lógica de negocio, controla el acceso a la base de datos junto a registry, es el modulo encargado de gestionar toda la información que produce cada módulo, cambios de estado, inactividad, incidencias, registrar auditorias (junto a registry en algún caso concreto), mandarle información al front y ciertas funciones de control con el resto de modulos.
- **EV_CP_M (Monitor):** gestiona el estado de engine compartiéndolo con central, ademas solicita el alta y baja con registry y la autenticacion con central.
- **EV_CP_E (Engine):** gestiona la lógica de un suministro
- **EV_Driver:** cliente que solicita los servicios de carga.
Esta separación coincide con la del enunciado, que distingue entre *aplicación central*, *aplicación del punto de recarga* (con dos funciones) y *aplicación del conductor*.

- **EV_Registry:** Da de alta a los CPs (guarda sus dato en la bd y proporción credenciales a los mismos para que se puedan autenticar en la bd).
- **EV_W (Weather Service):** proceso independiente que consulta OpenWeather (geocoding + current weather), mantiene un conjunto dinámico de ubicaciones y notifica a EV_Central cada 4 s mediante HTTP.

3.2 Canales de comunicación

- **Socket Monitor → Central:** para REGISTER y HEARTBEAT (1 s). Implementado en el hilo/loop de escucha de monitores de EV_Central y en el bucle de heartbeat_loop() de EV_CP_M.
- **Socket Monitor → Engine:** para comprobar salud y pasar órdenes locales.
- **Kafka Central ↔ Engine:** para autorización de sesiones y para telemetrías.
- **Kafka Central ↔ Driver:** para peticiones del conductor y devolución de ticket.
- **HTTP + WebSocket Central ↔ Panel:** para visualización en tiempo real.
- **HTTP EV_W → Central:** POST /weather/alert cada 4 s con {location, temp_c, alert} para control automático por clima.

3.3 Soporte web y persistencia

El proceso **EV_Central** incluye un servidor **FastAPI** que sirve el panel (`index.html`), una API REST mínima (`/cp`, `/central/summary`, `/command`) y un **WebSocket** para notificar cambios a los clientes. A su vez, persiste todo en **SQLite** usando el esquema definido en `schema.sql` y creado en `init_db.py`.

3.4 Justificación de la separación Monitor / Engine

Se ha separado el CP en **Monitor** y **Engine** para reflejar el modelo del enunciado (un proceso que se registra y otro que recibe órdenes) y, sobre todo, para que una caída del Engine no implique la desaparición del CP en la Central: el Monitor puede seguir enviando heartbeats con estado **KO**, y la Central puede reaccionar (STOP) y actualizar el panel.

4. Componentes y Protocolos

4.1. EV_Central (orquestración, API/WS, persistencia y sockets)

EV_Central concentra la lógica de negocio: recibe registros/latidos por **socket TCP** desde los monitores, autoriza servicios y publica comandos por **Kafka**, expone una **API HTTP** y un **WebSocket** para el panel, y persiste en **SQLite** (tablas: charging_points, sessions, events, drivers).

Arranque/uso (CLI):

```
1  """
2  EV_Central — Sistema central de monitorización de puntos de recarga
3  Uso:
4  ... python EV_Central.py <puerto_http> <ip_broker:puerto>
5
6  Ejemplo:
7  ... python EV_Central.py 8080 127.0.0.1:9092
8  """
```

Gestión de heartbeats (buffers y timeout):

```
# -----
# Monitoreo de heartbeats
# -----
LAST_HEARTBEAT = {}
HEARTBEAT_TIMEOUT = 3 # segundos sin recibir heartbeat → DESCONECTADO

# Última telemetría por CP para exponerla en /cp y en el panel
LAST_TELEMETRY: Dict[str, Dict[str, Any]] = {}

# ---- Estado y buffers de CENTRAL ----
CENTRAL_STATUS = "OK"
LAST_MESSAGES = deque(maxlen=5) # últimos 5 mensajes
RECENT_SESSIONS = deque(maxlen=50) # histórico corto de inicios de sesión
MAIN_LOOP = None
```

EV_Central por terminal:

```
(.venv) C:\Users\pablo\Desktop\Universidad\SD\ds1\EVCharging\EV_Central>python EV_Central.py 8080
0 localhost:9092 127.0.0.1:0
✚ Iniciando EV_Central ...
  • Puerto HTTP: 8080
  • Broker Kafka: localhost:9092
  • Puerto SOCKET monitores: 9000
  • Todos los puntos de recarga marcados como DESCONECTADOS al iniciar CENTRAL.
🔊 Escuchando monitores en puerto 9000...
INFO: Started server process [30644]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8080 (Press CTRL+C to quit)
📺 CP registrado desde monitor: CP-ALICANTE-01 (Alicante Centro, 0.45 €/kWh)
```

Lógica de clima:

```
@app.post("/weather/alert")
async def api_weather_alert(u: WeatherUpdate, request: Request):
    location = (u.location or "").strip()
    if not location:
        raise HTTPException(status_code=400, detail="location vacío")

    loc_key = norm_loc(location)

    prev = WEATHER_STATE.get(loc_key)
    prev_alert = None if prev is None else bool(prev.get("alert"))

    new_alert = bool(u.alert)
    temp_c = float(u.temp_c)

    # Actualiza siempre el estado (último valor conocido)
    WEATHER_STATE[loc_key] = {
        "location": location,
        "temp_c": temp_c,
        "alert": new_alert,
        "ts": now_iso()
    }

    src_ip = request.client.host if request.client else "unknown"

    # Panel: sí, siempre (cada 4s)
    await notify_panel({"type": "weather.update", **WEATHER_STATE[loc_key]})
```

```

# --- Aplicar efecto a CPS (idempotente) ---
if new_alert:
    for cp_id in cp_ids:
        row = get_cp_from_db(cp_id) or {}
        cur_status = (row.get("status") or "").upper()
        if is_protected(cur_status):
            continue
        WEATHER_DISABLED_CPS.add(cp_id)

        if cp_id in ACTIVE_SESSIONS:
            # está cargando -> stop pendiente (solo si no estaba ya)
            if cp_id not in WEATHER_PENDING_STOP:
                WEATHER_PENDING_STOP.add(cp_id)
                await notify_panel({"type": "weather.stop_pending", "cp_id": cp_id, "location": location})

                log_central_msg("WEATHER_STOP_PENDING", {"cp_id": cp_id, "location": location})
                insert_audit_log(
                    src_ip, "EV_W", "WEATHER_STOP_PENDING",
                    json.dumps({"cp_id": cp_id, "location": location}, ensure_ascii=False)
                )
            else:
                # STOP solo si no está ya PARADO
                if cur_status != "PARADO":
                    await send_cp_command("STOP", cp_id, source="weather.alert_on")
                    update_cp(cp_id, "PARADO")
                    await notify_panel({"type": "status", "cp_id": cp_id, "status": "PARADO"})

                    log_central_msg("WEATHER_STOP", {"cp_id": cp_id, "location": location})
                    insert_audit_log(
                        src_ip, "EV_W", "WEATHER_STOP",
                        json.dumps({"cp_id": cp_id, "location": location}, ensure_ascii=False)
                    )

```

4.2 EV_CP_M (Monitor del punto)

El **Monitor** da de alta el CP en CENTRAL por **socket** (**REGISTER**), emite **HEARTBEAT** cada 1 s, y vigila el **Engine** (por socket) para reportar salud **OK/KO**.

Uso/CLI y REGISTER hacia CENTRAL:

```

if len(sys.argv) < 6:
    print("Uso: python EV_CP_M.py <ip_engine:puerto> <ip_central:puerto> <id_cp> <location> <price>")
    sys.exit(1)

```

```

# -----
# Comunicación con CENTRAL (por socket)
# -----
def send_to_central(message):
    try:
        with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
            s.connect((CENTRAL_IP, CENTRAL_PORT))
            s.sendall(json.dumps(message).encode())
    except Exception as e:
        print(f"⚠ Error enviando a CENTRAL: {e}")

```

Bucle de Heartbeat:

```
# -----  
# Heartbeat loop  
# -----  
async def heartbeat_loop():  
    while True:  
        resp = ping_engine()  
        health = "OK" if resp == "OK" else "KO"  
        send_to_central({  
            "action": "HEARTBEAT",  
            "cp_id": CP_ID,  
            "health": health  
        })  
        print(f"💖 Heartbeat {CP_ID} ({health})")  
        await asyncio.sleep(1)
```

Cambio de ubicación en caliente:

```
# -----  
# ENGINE (socket)  
# -----  
def send_id_to_engine():  
    try:  
        with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:  
            s.settimeout(3)  
            s.connect((ENGINE_IP, ENGINE_PORT))  
            s.sendall(json.dumps({"cp_id": CP_ID, "location": LOCATION}).encode())  
            ack = s.recv(16).decode().strip()  
            return ack == "ACK"  
    except Exception as e:  
        print(f"Error al enviar ID al Engine: {e}")  
        return False
```

```

elif op == "5":
    try:
        if not is_authenticated_local():
            print("No autenticado. Primero opción 3 (Autenticar).")
            continue

        new_loc = (await asyncio.to_thread(input, "Nueva ubicación (ej: Oslo,NO): ")).strip()
        if not new_loc:
            print("Ubicación vacía, cancelado.")
            continue

        # Actualiza variable local para que a partir de ahora uses la nueva location en alta/auth/etc.
        current_location = new_loc.replace(" ", "_")

        # Pídeselo a Central (irá cifrado automáticamente porque action != AUTH)
        resp = send_to_central_and_rcv(
            {"action": "UPDATE_LOCATION", "cp_id": CP_ID, "location": current_location, "ip": ENGINE_IP},
            timeout=3
        )

        print("Central response:", resp)

        # (Opcional) si quieres que Engine también "se entere" (no es necesario para weather),
        # puedes reenviar {"cp_id", "location"} al engine:
        # send_id_to_engine()

    except Exception as e:
        print("Error:", e)

```

EV_Monitor por el terminal:

```

(.venv) C:\Users\pablo\Desktop\Universidad\SD\ds1\EVCharging\EV_CP_M>python EV_CP_M.py 127.0.0.1:6001
127.0.0.1:9000 CP-ALICANTE-01 "Alicante Centro" 0.45
🔍 EV_CP_M CP-ALICANTE-01 iniciado | Engine: 127.0.0.1:6001 | Central: 127.0.0.1:9000
✅ Engine confirmó ACK para CP-ALICANTE-01. Iniciando heartbeats...
🍷 Heartbeat CP-ALICANTE-01 (OK)
🍷 Heartbeat CP-ALICANTE-01 (OK)
🍷 Heartbeat CP-ALICANTE-01 (OK)
🍷 Heartbeat CP-ALICANTE-01 (OK)
🍷 Heartbeat CP-ALICANTE-01 (OK)

```

4.3 EV_CP_E (Engine del punto)

El **Engine** escucha **comandos** desde CENTRAL por **Kafka** (p. ej. `central.authorize`, `central.command`), simula la sesión de carga y publica **telemetrías** y evento de fin de sesión. Expone un **socket local** para que el Monitor le pase info y permite simular estado **KO/OK**.

Uso/CLI:

```
# -----  
# ARGUMENTOS  
# -----  
if len(sys.argv) < 3:  
    print("Uso: python EV_CP_E.py <broker_ip:puerto> <puerto_engine>")  
    sys.exit(1)  
  
BROKER = sys.argv[1]  
ENGINE_PORT = int(sys.argv[2])
```

Simulación OK/KO (hilo de control por teclado):

```
# -----  
# CONTROL DESDE TECLADO (para simular KO/OK)  
# -----  
def control_thread():  
    global HEALTH  
    while True:  
        try:  
            cmd = input("").strip().upper()  
        except EOFError:  
            break  
        if cmd == "KO":  
            HEALTH = "KO"  
            print("⚠ Engine simulado en estado KO.")  
        elif cmd == "OK":  
            HEALTH = "OK"  
            print("✅ Engine vuelve a estado OK.")
```

Central.authorize:

```
if msg.topic == "central.authorize":
    # 1) Solo si es para este CP
    if data.get("cp_id") != CP_ID:
        continue
    # 2) No aceptes authorize si el CP no está ACTIVADO o está en STOP
    if STOP.is_set() or STATUS != "ACTIVADO":
        print("🚫 Ignoro authorize: CP no disponible (PARADO/NO ACTIVO).")
        continue
    # 3) Evitar duplicar sesiones si ya está suministrando (por si llegan dobles)
    if STATUS == "SUMINISTRANDO" or (CHARGE_TASK and not CHARGE_TASK.done()):
        print("📌 Ya estaba suministrando; ignoro authorize duplicado.")
        continue

    print("🔔 Autorizado suministro por CENTRAL")
    CHARGE_TASK = asyncio.create_task(start_charging(producer))
```

Central.command:

```
elif msg.topic == "central.command":
    action = data.get("action", "").upper()
    target = data.get("cp_id", "ALL")
    print(f"⚙️ Orden recibida desde CENTRAL: {action} → {target}")

    # aquí aceptamos global o dirigido a este CP
    if target != "ALL" and target != CP_ID:
        continue

    if action == "STOP":
        STOP.set()
        if CHARGE_TASK and not CHARGE_TASK.done():
            pass
        await send_status(producer, "PARADO")
        print(f"🛑 Carga detenida por CENTRAL en {CP_ID}")

    elif action == "RESUME":
        STOP.clear()
        await send_status(producer, "ACTIVADO")
        print(f"▶️ CP {CP_ID} reanudado por CENTRAL")
```

Prueba por terminal (EV_CP_E):

```
(.env) C:\Users\pablo\Desktop\Universidad\SD\ds1\EVcharging\EV_CP_E>python EV_CP_E.py 127.0.0.1:9092 6001
EV_CP_E conectado a Kafka 127.0.0.1:9092
Puerto del Engine para monitor: 6001
Escribe 'KO' o 'OK' para simular el estado del engine.
Esperando mensajes de CENTRAL...
Escuchando conexiones del monitor en ('0.0.0.0', 6001)...
Recibido ID CP-ALICANTE-01 y ubicación Alicante Centro
Registrando CP CP-ALICANTE-01 en CENTRAL (Alicante Centro) - 0.52 €/kWh
Estado actualizado: ACTIVADO
KO
Engine simulado en estado KO.
OK
Engine vuelve a estado OK.
```

Prueba carga en un CP:

```
KO
Engine simulado en estado KO.
OK
Engine vuelve a estado OK.
Autorizado suministro por CENTRAL
Estado actualizado: SUMINISTRANDO
Iniciando carga en CP-ALICANTE-01 (Precio: 0.52 €/kWh)
CP-ALICANTE-01: 6.68 kW - 0.579 €
CP-ALICANTE-01: 6.31 kW - 1.126 €
CP-ALICANTE-01: 7.08 kW - 1.739 €
CP-ALICANTE-01: 7.34 kW - 2.376 €
CP-ALICANTE-01: 7.12 kW - 2.993 €
CP-ALICANTE-01: 6.9 kW - 3.591 €
CP-ALICANTE-01: 6.06 kW - 4.116 €
CP-ALICANTE-01: 7.11 kW - 4.732 €
CP-ALICANTE-01: 7.21 kW - 5.357 €
CP-ALICANTE-01: 7.43 kW - 6.001 €
Estado actualizado: ACTIVADO
Carga finalizada.
```

4.4 EV_Driver (Cliente solicitante)

El **Driver** envía peticiones **driver.request** a CENTRAL, consume **driver.update** y **driver.telemetry**, y respeta **4 s** entre servicios cuando procesa un fichero de CPs.

Uso/CLI y lectura de fichero de CPs:

```
USO = "Uso: python EV_Driver.py <broker_ip:puerto> <id_cliente> [fichero_cp_ids]"
```

```
if len(sys.argv) >= 4:
    with open(sys.argv[3], "r", encoding="utf-8") as f:
        cp_ids = []
        for line in f:
            t = line.strip()
            if not t or t.startswith("#"):
                continue
            cp_ids.append(t)
else:
    cp = input("CP a solicitar: ").strip()
    cp_ids = [cp]
```

```
# 1) Enviar solicitud a CENTRAL
await producer.send_and_wait(
    "driver.request",
    json.dumps({
        "cp_id": cp,
        "driver_id": driver_id,
        "request_id": req_id
    }).encode()
)
print(f"📄 Solicitud enviada → CP={cp} | request_id={req_id}")
```

```
# 3) Pausa de 4 s entre solicitudes (requisito de la práctica)
await asyncio.sleep(4)
```

Prueba por terminal (EV_Driver):

```
(.venv) C:\Users\pablo\Desktop\Universidad\SD\ds1\EVCharging\EV_Driver>python EV_Driver.py localhost:9092 user-123
CP a solicitar: CP-ALICANTE-01
🚗 EV_Driver listo → broker=localhost:9092 | driver=user-123 | servicios=1
📄 Solicitud enviada → CP=CP-ALICANTE-01 | request_id=33611611-eb36-4d0c-ba0d-e4d1512f6846
🔔 Update: AUTHORIZED - Autorizado en CP-ALICANTE-01
⚡ CP-ALICANTE-01 → 6.68 kW | 1.1130 kWh | 0.58 €
⚡ CP-ALICANTE-01 → 6.31 kW | 2.1650 kWh | 1.13 €
⚡ CP-ALICANTE-01 → 7.08 kW | 3.3450 kWh | 1.74 €
⚡ CP-ALICANTE-01 → 7.34 kW | 4.5680 kWh | 2.38 €
⚡ CP-ALICANTE-01 → 7.12 kW | 5.7550 kWh | 2.99 €
⚡ CP-ALICANTE-01 → 6.9 kW | 6.9050 kWh | 3.59 €
⚡ CP-ALICANTE-01 → 6.06 kW | 7.9150 kWh | 4.12 €
⚡ CP-ALICANTE-01 → 7.11 kW | 9.1000 kWh | 4.73 €
⚡ CP-ALICANTE-01 → 7.21 kW | 10.3020 kWh | 5.36 €
⚡ CP-ALICANTE-01 → 7.43 kW | 11.5400 kWh | 6.00 €
🔔 Update: FINISHED - Servicio finalizado en CP-ALICANTE-01
✅ Carga finalizada en CP-ALICANTE-01 → 11.5400 kWh | 6.000 € (reason=ENDED)
```

4.5 EV_W (Servicio Weather)

EV_W es un proceso que permite configurar y modificar en tiempo real el conjunto de ubicaciones (ciudades) monitorizadas. Cada 4 segundos consulta la temperatura usando OpenWeather y notifica a EV_Central mediante `POST /weather/alert`.

Para soportar nombres de ciudad, EV_W usa el **Direct Geocoding API** de OpenWeather para obtener coordenadas (lat/lon) a partir de `City, CountryCode` y cachea el resultado para no repetir geocoding en cada ciclo. Con esas coordenadas consulta el endpoint de **Current Weather Data** con unidades métricas.

EV_W mantiene un menú interactivo para listar/añadir/eliminar ubicaciones sin reiniciar, cumpliendo el requisito de modificación “a voluntad”.

4.6 EV_Registry

Es el módulo encargado de dar de alta y autenticar los CPs. También de dar de baja los mismos CPs. Funciona mediante API REST https, se encarga de guardar en la base de datos los CPs al darse de alta.

4.7 Protocolos y tópicos (resumen)

Sockets TCP

- Monitor → Central: `REGISTER, HEARTBEAT(OK|KO)`
- Monitor ↔ Engine: latido local / control simple

Kafka

- Driver → Central: `driver.request`
- Central → Driver: `driver.update, driver.telemetry`
- Central → Engine: `central.authorize, central.command`
- Engine → Central: `cp.telemetry, cp.session_ended`

HTTP/WS

- Weather → Central: `HTTP POST /weather/alert` cada 4 segundos.
- Panel: `GET / (HTML), GET /cp, GET /central/summary, POST /command, WS /ws`

⚡ EV Central - Panel de Monitorización ⚡

● STOP ALL

▶ RESUME ALL

ID	Ubicación	Precio (€/kWh)	Estado	kWh total	€ total	Driver	Última actualización	Acciones
CP-ALICANTE-01	Alicante Centro	0.52	DESCONECTADO	—	—	—	19:07:07	<button>STOP</button> <button>RESUME</button>
CP-ALICANTE-02	Desconocida	0.30	PARADO	—	—	—	19:07:22	<button>STOP</button> <button>RESUME</button>
CP-ALICANTE-03	Alicante Este	0.52	ACTIVADO	—	—	—	19:07:32	<button>STOP</button> <button>RESUME</button>

Sesiones iniciadas

Inicio (fecha/hora)	Driver	CP	Session ID
2025-11-06T18:50:01	user-123	CP-ALICANTE-01	38

Últimos 5 mensajes

Fecha/Hora	Tipo	Detalle
2025-11-06T19:05:44	DISCONNECTED	{"cp_id":"CP-ALICANTE-03","since_sec":3}
2025-11-06T19:05:49	REGISTER	{"cp_id":"CP-ALICANTE-03","location":"Alicante Este","price":0.49}
2025-11-06T19:05:49	REGISTER	{"cp_id":"CP-ALICANTE-03","location":"Alicante Este","price":0.52}

ID	Ubicación	Precio (€/kWh)	Estado	kWh total	€ total	Driver	Última actualización	Acciones
CP-ALICANTE-01	Alicante Centro	0.52	DESCONECTADO	—	—	—	19:07:07	<button>STOP</button> <button>RESUME</button>
CP-ALICANTE-02	Desconocida	0.30	PARADO	—	—	—	19:07:22	<button>STOP</button> <button>RESUME</button>
CP-ALICANTE-03	Desconocida	0.30	SUMINISTRANDO	3.320	1.73	user-124	19:08:36	<button>STOP</button> <button>RESUME</button>

Sesiones iniciadas

Inicio (fecha/hora)	Driver	CP	Session ID
2025-11-06T19:08:34	user-124	CP-ALICANTE-03	39
2025-11-06T18:50:01	user-123	CP-ALICANTE-01	38

ID	Ubicación	Precio (€/kWh)	Estado	kWh total	€ total	Driver	Última actualización	Acciones
CP-ALICANTE-01	Alicante Centro	0.52	DESCONECTADO	—	—	—	19:07:07	<button>STOP</button> <button>RESUME</button>
CP-ALICANTE-02	Alicante Norte	0.43	ACTIVADO	—	—	—	19:34:48	<button>STOP</button> <button>RESUME</button>
CP-ALICANTE-03	Alicante Este	0.52	AVERIA	—	—	—	19:34:47	<button>STOP</button> <button>RESUME</button>

5. Flujos principales

5.1 Registro del CP y heartbeats (Registry ↔ Monitor ↔ Central // Monitor ↔ Engine)

1. Al arrancar, el **Monitor** se conecta con **Engine** por sockets enviando location id y precio kw
2. En el menu podemos autenticar y le mandará a registry la informacion para que le de de alta y le de las credenciales.

```
===== MENÚ CP_MONITOR =====
1) Dar de alta (Registry REST)
2) Dar de baja (Registry REST)
3) Autenticar (Central SOCKET)
4) Parar Heartbeats
5) Cambiar ubicación (UPDATE en Central)
0) Salir
=====

Opción: 1
Alta OK. Credential guardada en cp_2_credential.json
```

3. Luego podemos autenticar y monitor le pasará la credential a central la cuál le devolverá la secret key (por socket) que cifrará todos los mensajes mandandos a central tanto d engine como de monitor.
4. Comienzan los heartbeats por socket con central

Monitor → Registry:

```
def registry_alta(location: str):
    url = f"{REGISTRY_BASE}/cp/{CP_ID}"
    payload = {"cp_id": CP_ID, "location": location, "price": PRICE, "ip": ENGINE_IP}
    r = requests.put(url, json=payload, verify=REGISTRY_VERIFY, timeout=5)
    if r.status_code != 200:
        raise RuntimeError(f"ALTA falló: {r.status_code} {r.text}")
    data = r.json()
    cred = data["credential"]
    with open(CRED_FILE, "w", encoding="utf-8") as f:
        json.dump({"cp_id": CP_ID, "credential": cred}, f)
    print("Alta OK. Credential guardada en", CRED_FILE)
```

Registry → Monitor: manda credential

```

@app.put("/cp/{cp_id}")
def alta_cp(cp_id: str, payload: AltaReq):
    cp_id = cp_id.strip()
    if payload.cp_id.strip() != cp_id:
        raise HTTPException(status_code=400, detail="cp_id del path y body no coinciden")

    upsert_cp(cp_id, payload.location.strip(), float(payload.price), payload.ip.strip())
    insert_audit_log(payload.ip.strip(), "CP_M_"+cp_id, "ALTA_CP", f"CP dado de alta correctamente -> Location: {payload.location}, Price: {payload.price}")
    cred = issue_credential(cp_id)

    return {
        "ok": True,
        "cp": get_cp(cp_id),
        "credential": cred
    }

```

Monitor → Engine: envío de ID y espera ACK

```

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.settimeout(3)
    s.connect((ENGINE_IP, ENGINE_PORT))
    s.sendall(json.dumps({"cp_id": CP_ID, "location": LOCATION}).encode())
    ack = s.recv(16).decode().strip()
    return ack == "ACK"

```

Monitor → Central: HEARTBEAT cada 1 s

```

print(f"💖 Heartbeat {CP_ID} ({health})")
await asyncio.sleep(1)

```

Central: servidor de sockets (REGISTER/HEARTBEAT) + STOP automático si KO en SUMINISTRANDO

```

elif action == "HEARTBEAT":
    LAST_HEARTBEAT[cp_id] = loop.time()
    health = (msg.get("health") or "KO").upper()
    new_status = "ACTIVADO" if health == "OK" else "AVERIA"

    # Si el CP entra en KO mientras está SUMINISTRANDO → STOP inmediato
    try:
        cur = get_cp_from_db(cp_id)
        cur_status = (cur.get("status") if cur else "DESCONECTADO").upper()
    except Exception:
        cur_status = "DESCONECTADO"

    if health == "KO" and cur_status == "SUMINISTRANDO":
        # Publica STOP para ese CP vía Kafka (desde este hilo: run_coroutine_threadsafe)
        asyncio.run_coroutine_threadsafe(
            kafka_producer.send_and_wait(
                "central.command",
                json.dumps({"action": "STOP", "cp_id": cp_id}).encode()
            ),
            loop
        )

```

```
(.venv) C:\Users\pablo\Desktop\Universidad\SD\ds1\EVCharging\EV_CP_M>python EV_CP_M.py 127.0.0.1:6002 127.0.0.1:9000 CP-ALICANTE-02 "Alicante Norte" 0.30
EV_CP_M CP-ALICANTE-02 iniciado | Engine: 127.0.0.1:6002 | Central: 127.0.0.1:9000
✅ Engine confirmó ACK para CP-ALICANTE-02. Iniciando heartbeats...
❤️ Heartbeat CP-ALICANTE-02 (OK)
❤️ Heartbeat CP-ALICANTE-02 (OK)
❤️ Heartbeat CP-ALICANTE-02 (OK)
❤️ Heartbeat CP-ALICANTE-02 (OK)
❤️ Heartbeat CP-ALICANTE-02 (OK)
❤️ Heartbeat CP-ALICANTE-02 (OK)
❤️ Heartbeat CP-ALICANTE-02 (KO)
❤️ Heartbeat CP-ALICANTE-02 (KO)
❤️ Heartbeat CP-ALICANTE-02 (KO)
❤️ Heartbeat CP-ALICANTE-02 (KO)
```

5.2 Solicitud del Driver y autorización (Driver → Central → Engine) + telemetrías y ticket final

Descripción del flujo:

1. **Driver** publica `driver.request(cp_id, driver_id, request_id)`.
2. **Central** valida disponibilidad y **autoriza** al CP enviando `central.authorize`. Además, notifica al **Driver** con `driver.update: AUTHORIZED`.
3. **Engine** inicia la carga, publica **telemetrías** periódicas (`cp.telemetry`).
4. **Central** reenvía telemetrías a **Driver** (`driver.telemetry`), actualiza panel y persiste progreso.
5. Al terminar, **Engine** envía fin de sesión; **Central** cierra, actualiza estado y manda **ticket FINISHED** al **Driver**.

Driver: envía request + espera y muestra FINISHED / telemetrías:

```
# 1) Enviar solicitud a CENTRAL
await producer.send_and_wait(
    "driver.request",
    json.dumps({
        "cp_id": cp,
        "driver_id": driver_id,
        "request_id": req_id
    }).encode()
)
print(f"📄 Solicitud enviada → CP={cp} | request_id={req_id}")
```

```

elif msg.topic == "driver.telemetry":
    # Progreso en tiempo real
    kw = data.get("kw")
    kwh_total = data.get("kwh_total")
    eur_total = data.get("eur_total")
    try:
        print(f" ⚡ {cp} → {kw} kW | {kwh_total:.4f} kWh | {eur_total:.2f} €")
    except Exception:
        print(f" ⚡ {cp} → {kw} kW | {kwh_total} kWh | {eur_total} €")

```

```

if status == "FINISHED":
    # Ticket final (resumen)
    summary = data.get("summary", {}) or {}
    kwh = summary.get("kwh")
    eur = summary.get("amount_eur")
    reason = summary.get("reason", "ENDED")
    try:
        print(f" ✅ Carga finalizada en {cp} → {kwh:.4f} kWh | {eur:.3f} € (reason={reason})")
    except Exception:
        print(f" ✅ Carga finalizada en {cp} → {kwh} kWh | {eur} € (reason={reason})")
    break

```

Engine: al recibir **central.authorize** inicia carga:

```

if msg.topic == "central.authorize":
    # 1) Solo si es para este CP
    if data.get("cp_id") != CP_ID:
        continue
    # 2) No aceptes authorize si el CP no está ACTIVADO o está en STOP
    if STOP.is_set() or STATUS != "ACTIVADO":
        print(" 🛑 Ignoro authorize: CP no disponible (PARADO/NO ACTIVO).")
        continue
    # 3) Evitar duplicar sesiones si ya está suministrando (por si llegan dobles)
    if STATUS == "SUMINISTRANDO" or (CHARGE_TASK and not CHARGE_TASK.done()):
        print(" ⓘ Ya estaba suministrando; ignoro authorize duplicado.")
        continue

    print(" 🛎 Autorizado suministro por CENTRAL")
    CHARGE_TASK = asyncio.create_task(start_charging(producer))

```

Engine: telemetrías periódicas → **cp.telemetry**:

```

telem = {
    "cp_id": CP_ID,
    "session_id": 1, # ID fijo de demo
    "kw": kw,
    "kwh_total": round(kwh, 3),
    "eur_total": amount
}
await producer.send_and_wait("cp.telemetry", json.dumps(telem).encode())
print(f" 🟢 {CP_ID}: {kw} kW — {amount:.3f} €")

```

Panel: al recibir **telemetry** fuerza estado SUMINISTRANDO:

```
// Telemetría → fuerza SUMINISTRANDO + totales + driver
if (msg.type === "telemetry") {
  updateRow({
    cp_id: msg.cp_id,
    status: "SUMINISTRANDO",
    kwh_total: msg.kwh_total,
    eur_total: msg.eur_total,
    driver_id: msg.driver_id
  });
  return;
}
```

5.3 Comandos de Central (STOP / RESUME) a uno o a todos los CP

Descripción del flujo:

1. Un usuario pulsa **STOP** o **RESUME** en el panel (o vía API `/command`).
2. **Central** publica en Kafka `central.command {action, cp_id}`.
3. **Engine** que corresponda recibe la orden: si **STOP**, marca **PARADO** y detiene la carga; si **RESUME**, vuelve a **ACTIVADO**.

Central: endpoint `/command` → publica en `central.command`

```
class Command(BaseModel):
    action: str = "STOP" | "RESUME"
    cp_id: str = "ALL" # puedes enviar "ALL" o un ID concreto

@app.post("/command")
async def send_command(cmd: Command):
    payload = {"action": cmd.action.upper(), "cp_id": cmd.cp_id}
    await kafka_producer.send_and_wait("central.command", json.dumps(payload).encode())
    return {"status": "ok", "sent": payload}
```

Engine: maneja **central.command**

```
elif msg.topic == "central.command":
    action = data.get("action", "").upper()
    target = data.get("cp_id", "ALL")
    print(f"⚙️ Orden recibida desde CENTRAL: {action} → {target}")

    # aquí aceptamos global o dirigido a este CP
    if target != "ALL" and target != CP_ID:
        continue

    if action == "STOP":
        STOP.set()
        if CHARGE_TASK and not CHARGE_TASK.done():
            pass
        await send_status(producer, "PARADO")
        print(f"🛑 Carga detenida por CENTRAL en {CP_ID}")

    elif action == "RESUME":
        STOP.clear()
        await send_status(producer, "ACTIVADO")
        print(f"▶️ CP {CP_ID} reanudado por CENTRAL")
```

5.4 Cierre de sesión y persistencia (Central)

Durante la sesión, **Central** actualiza progreso en **sessions** (**kwh**, **amount_eur**) y mantiene buffers para el panel. Al final, cambia el estado del CP, cierra la sesión y envía ticket **FINISHED** al Driver.

Helpers de BBDD usados durante el flujo

```
def start_session(cp_id, driver_id, price_eur_kwh):
    with closing(get_db()) as con:
        cur = con.cursor()
        cur.execute(
            "INSERT INTO sessions (cp_id, driver_id, price_eur_kwh, status) VALUES (?, ?, ?, 'RUNNING')",
            (cp_id, driver_id, price_eur_kwh)
        )
        sid = cur.lastrowid
        con.commit()
        return sid

def update_session_progress(session_id, kwh, amount_eur):
    with closing(get_db()) as con:
        con.execute(
            "UPDATE sessions SET kwh=?, amount_eur=? WHERE id=?",
            (kwh, amount_eur, session_id)
        )
        con.commit()

def end_session(session_id, kwh, amount_eur, ended_status="ENDED"):
    with closing(get_db()) as con:
        con.execute(
            "UPDATE sessions SET ended_at=CURRENT_TIMESTAMP, kwh=?, amount_eur=?, status=? WHERE id=?",
            (kwh, amount_eur, ended_status, session_id)
        )
        con.commit()
```

API del panel

```
@app.get("/cp")
def api_list_cps():
    ... cps = list_cps()
    ... for cp in cps:
    ...     lt = LAST_TELEMETRY.get(cp["id"])
    ...     if lt:
    ...         cp["kwh_total"] = lt.get("kwh_total", 0.0)
    ...         cp["eur_total"] = lt.get("eur_total", 0.0)
    ...         cp["driver_id"] = lt.get("driver_id")
    ...     else:
    ...         cp["kwh_total"] = None
    ...         cp["eur_total"] = None
    ...         cp["driver_id"] = None
    ... return cps

@app.websocket("/ws")
async def websocket_endpoint(ws: WebSocket):
    ... await ws.accept()
    ... PANEL_CLIENTS.add(ws)
    ... print("👤 Cliente conectado al panel")
    ... try:
    ...     while True:
    ...         await ws.receive_text()
    ... except WebSocketDisconnect:
    ...     PANEL_CLIENTS.discard(ws)
    ...     print("👤 Cliente desconectado")

@app.get("/central/summary")
def api_central_summary():
    ... return {
    ...     "status": CENTRAL_STATUS,
    ...     "messages": list(LAST_MESSAGES),
    ...     "sessions": list(RECENT_SESSIONS)
    ... }
```

5.5 Detección de desconexión (timeout de heartbeat)

Si **Central** no recibe **HEARTBEAT** de un CP en **HEARTBEAT_TIMEOUT** segundos, actualiza el estado del CP a **DESCONECTADO**, notifica al panel y registra el evento.

Constante de timeout + tarea periódica

```
# -----
# Monitoreo de heartbeats
# -----

LAST_HEARTBEAT = {}
HEARTBEAT_TIMEOUT = 3 # segundos sin recibir heartbeat → DESCONECTADO

async def monitor_disconnections():
    """ Marca CPs como DESCONECTADOS si no envían heartbeats recientes. """
    while True:
        now = asyncio.get_running_loop().time()
        for cp_id, last in list(LAST_HEARTBEAT.items()):
            if now - last > HEARTBEAT_TIMEOUT:
                print(f"▲ CP {cp_id} no ha mandado heartbeat en {HEARTBEAT_TIMEOUT}s → DESCONECTADO")
                update_cp(cp_id, "DESCONECTADO")
                cp_data = get_cp_from_db(cp_id)
                if cp_data:
                    cp_data["status"] = "DESCONECTADO"
                    await notify_panel({"type": "heartbeat", **cp_data})
                    log_central_msg("DISCONNECTED", {"cp_id": cp_id, "since_sec": HEARTBEAT_TIMEOUT})
                    del LAST_HEARTBEAT[cp_id]

        await asyncio.sleep(2)
```

5.6 Control automático por clima (EV_W → Central → CPs)

1. EV_W mantiene una lista dinámica de ubicaciones (ej. `Oslo`, `NO`, `Alicante`, `ES`).
2. Cada 4 s consulta OpenWeather y calcula `alert = (temp_c < 0)`.
3. EV_W envía `POST /weather/alert` a Central con `location`, `temp_c`, `alert`.
4. Central actualiza el estado meteorológico por ubicación y lo notifica al panel vía WebSocket.
5. Si `alert=True`, Central identifica los CPs cuya `location` coincide y aplica:
 - si el CP está SUMINISTRANDO, marca `pending_stop` y detiene al finalizar la sesión;
 - si no está suministrando, publica `STOP` y fuerza `PARADO` en BD.
 - no sobrescribe estados protegidos (`DESACTIVADO`, `AVERIA`, `DESCONECTADO`).
6. Si `alert=False`, Central elimina el bloqueo por clima y publica `RESUME`; si el CP estaba `PARADO` por clima, lo sube a `ACTIVADO`.

5.7 Cambio de ubicación en caliente (Monitor → Central)

El Monitor incorpora una opción para actualizar la ubicación de un CP durante la ejecución, sin reiniciar procesos y sin perder heartbeats. El monitor envía un mensaje `UPDATE_LOCATION` a Central; Central actualiza la BBDD y, opcionalmente, re-evalúa el estado meteorológico de la nueva ubicación para que el efecto sea inmediato (parada o reanudación según el clima).

6. Panel de monitorización

Panel web servido por **EV_Central** (HTTP) con actualizaciones en tiempo real por **WebSocket**. Muestra una tabla principal con **ID, ubicación, precio, estado, kWh/€, driver, última actualización y acciones STOP/RESUME**, y dos tablas auxiliares: **Sesiones iniciadas (máx. 50)** y **Últimos 5 mensajes**.

Además de estados y sesiones, el panel muestra el estado meteorológico por ubicación (última temperatura, alerta y timestamp), actualizado en tiempo real por WebSocket mediante eventos `weather.update`, y refleja automáticamente los cambios de estado del CP provocados por clima.

 EV Central - Panel de Monitorización 

STOP ALL

RESUME ALL

ID	Ubicación	Precio (€/kWh)	Estado	kWh total	€ total	Driver	Última actualización	Acciones	Registrado	Clave
1	Alicante,ES	0.45	ACTIVADO	—	—	—	19:11:57	STOP RESUME REVOKE KEY	ALTA	ACTIVA
2	Oslo,NO	0.50	PARADO	—	—	—	19:11:57	STOP RESUME REVOKE KEY	ALTA	ACTIVA
3	Barcelona,ES	0.49	ACTIVADO	—	—	—	19:11:57	STOP RESUME REVOKE KEY	ALTA	ACTIVA
4	Madrid,ES	0.35	ACTIVADO	—	—	—	19:11:57	STOP RESUME REVOKE KEY	ALTA	ACTIVA
5	Elche,ES	0.43	ACTIVADO	—	—	—	19:11:57	STOP RESUME REVOKE KEY	ALTA	ACTIVA

Suministros activos

Inicio (fecha/hora)	Driver	CP	Session ID
---------------------	--------	----	------------

 Weather (EV_W)

Ubicación	Temp (°C)	Alerta	Última actualización
Valencia,ES	10.46	OK (>=0°C)	2026-01-07T19:12:14
Alicante,ES	10.94	OK (>=0°C)	2026-01-07T19:12:14
Oslo,NO	-6.89	ALERTA (<0°C)	2026-01-07T19:12:14
Madrid,ES	5.15	OK (>=0°C)	2026-01-07T19:12:14
Moscow,RU	-3.88	ALERTA (<0°C)	2026-01-07T19:12:14
Elche,ES	10.55	OK (>=0°C)	2026-01-07T19:12:14

CPs deshabilitados por clima: 2
CPs con stop pendiente: —

Últimos 5 mensajes

Fecha/Hora	Tipo	Detalle
2026-01-07T19:10:05	AUTH_SUCCESS	{ "cp_id": "1", "location": "Alicante,ES", "price": 0.45 }
2026-01-07T19:07:42	DISCONNECTED	{ "cp_id": "1", "since_sec": 3.2 }
2026-01-07T19:07:38	KEY_REVOKED	{ "cp_id": "1" }
2026-01-07T19:07:03	WEATHER_STOP	{ "cp_id": "2", "location": "Oslo,NO" }
2026-01-07T19:07:03	COMMAND_SENT	{ "source": "weather.alert_on", "action": "STOP", "cp_id": "2" }

Estado de CENTRAL: OK

7. Mecanismos de cifrado

En la práctica se han implementado distintos mecanismos de cifrado y protección de la información con el objetivo de garantizar la **confidencialidad, integridad y autenticidad** de las comunicaciones entre los diferentes módulos del sistema EVCharging.

7.1 Cifrado del canal entre EV_CP_M y EV_Registry (HTTPS)

La comunicación entre el módulo **EV_CP_M (Monitor del Punto de Carga)** y **EV_Registry** se realiza mediante un **canal seguro HTTPS**, utilizando certificados **X.509** autofirmados.

- Se emplea **TLS** como mecanismo de cifrado del canal.
- Los certificados (**cert.pem** y **key.pem**) se generan mediante **OpenSSL**.
- El certificado incluye:
 - Clave RSA de 2048 bits
 - Hash SHA-256
 - Extensión **Subject Alternative Name (SAN)** para IP y DNS

Este mecanismo garantiza:

- Cifrado de los datos en tránsito
- Protección frente a ataques Man-in-the-Middle (MITM)
- No exposición de credenciales en claro

7.2 Protección segura de credenciales de los CPs

Tras el proceso de alta en el **EV_Registry**, cada CP recibe una **credencial secreta** que será utilizada posteriormente para autenticarse en **EV_Central**.

Almacenamiento seguro en Registry y Central

Las credenciales **no se almacenan en claro**, sino que se protegen mediante:

- Algoritmo: **PBKDF2-HMAC-SHA256**
- Uso de:
 - Salt aleatorio único por CP
 - Pepper compartido entre Registry y Central
 - 200.000 iteraciones

Esto impide:

- Ataques por fuerza bruta
- Ataques de rainbow tables

- Compromiso de credenciales si la base de datos es expuesta

7.3 Autenticación segura entre EV_CP_M y EV_Central

La autenticación entre el Monitor (**EV_CP_M**) y la Central (**EV_Central**) se realiza mediante **sockets TCP**, pero con verificación segura de credenciales.

Proceso:

1. El CP envía su **cp_id** y credencial obtenida del Registry.
2. EV_Central valida la credencial comparando hashes (HMAC seguro).
3. Si la autenticación es correcta:
 - EV_Central genera una **clave simétrica única por CP**
 - Dicha clave se envía al CP
 - La clave queda registrada en Central

Esta clave simétrica será usada posteriormente para el cifrado de mensajes.

7.4 Cifrado de mensajes entre CPs (Engine/Monitor) y EV_Central

Una vez autenticado el CP, **todos los mensajes enviados a EV_Central** se cifran utilizando **cifrado simétrico AES-GCM**.

Algoritmo utilizado

- **AES-GCM (Galois/Counter Mode)**
- Clave de 256 bits derivada mediante SHA-256
- Nonce aleatorio de 96 bits
- Autenticación de mensajes mediante **AAD (Additional Authenticated Data)**

Contenido del AAD

- Identificador del CP (**cp_id**)
- Marca temporal (**timestamp**)

Esto permite:

- Verificar la integridad del mensaje
- Evitar suplantación de CPs
- Garantizar autenticidad del origen

7.5 Protección frente a ataques de repetición (Replay Attacks)

Para evitar ataques de repetición, cada mensaje cifrado incluye:

- Timestamp
- Nonce único

La Central mantiene un registro temporal de nonces ya utilizados y rechaza cualquier mensaje que:

- Reutilice un nonce
- Tenga un timestamp fuera de una ventana temporal válida

7.6 Revocación de claves de cifrado

EV_Central implementa un mecanismo de **revocación de claves simétricas**, simulando una respuesta ante incidentes de seguridad.

- La Central puede borrar la clave de un CP específico.
- El CP queda automáticamente fuera de servicio.
- El CP debe volver a autenticarse para obtener una nueva clave.

Este mecanismo permite:

- Rotación de claves
- Mitigación ante compromisos de seguridad
- Cumplimiento de buenas prácticas de seguridad

8. Modelo de datos

Tablas principales en SQLite: `charging_points` (estado y precio), `drivers`, `sessions` (progreso, importe y estado) y `events` (audit trail). Índices en `sessions(cp_id)` y `events(cp_id)`.

```

-- Tabla de puntos de recarga (Charging Points)
CREATE TABLE IF NOT EXISTS charging_points (
    id TEXT PRIMARY KEY,
    location TEXT NOT NULL,
    price_eur_kwh REAL NOT NULL CHECK (price_eur_kwh >= 0),
    status TEXT NOT NULL DEFAULT 'DESCONECTADO', --- ACTIVADO, PARADO, SUMINISTRANDO, AVERIADO, DESCONECTADO
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- Tabla de conductores (Drivers)
CREATE TABLE IF NOT EXISTS drivers (
    id TEXT PRIMARY KEY,
    display_name TEXT NOT NULL
);

-- Tabla de sesiones de carga
CREATE TABLE IF NOT EXISTS sessions (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    cp_id TEXT NOT NULL REFERENCES charging_points(id),
    driver_id TEXT NOT NULL REFERENCES drivers(id),
    started_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    ended_at DATETIME,
    price_eur_kwh REAL NOT NULL,
    kwh REAL NOT NULL DEFAULT 0,
    amount_eur REAL NOT NULL DEFAULT 0,
    status TEXT NOT NULL DEFAULT 'RUNNING' --- RUNNING | ENDED | ABORTED | FAILED
);

```

```

-- Tabla de eventos (histórico)
CREATE TABLE IF NOT EXISTS events (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    ts DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    cp_id TEXT,
    driver_id TEXT,
    type TEXT NOT NULL, --- HEARTBEAT | STATUS | TELEMETRY | COMMAND | AUTH | ERROR
    payload TEXT NOT NULL
);

```

```

EVCharging > EV_Central > init_db.py > ...
1  import sqlite3
2
3  with open("schema.sql", "r", encoding="utf-8") as f:
4      schema = f.read()
5
6  conn = sqlite3.connect("evcentral.db")
7  conn.executescript(schema)
8  conn.commit()
9  conn.close()
10
11  print("✅ Base de datos 'evcentral.db' creada correctamente.")
12

```

8.1 Registro de auditoría

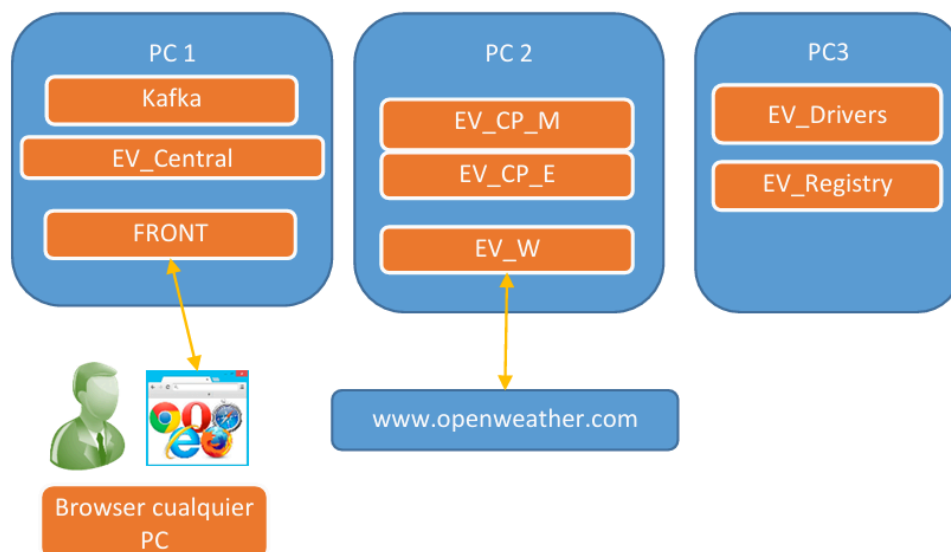
Se implementa un registro de auditoría persistente donde cada evento relevante del sistema queda almacenado de forma estructurada, incluyendo:

- a) fecha/hora del evento;
- b) origen (IP) y actor que lo provoca (CP_M, Driver, EV_W, Panel/Central);
- c) acción realizada (AUTH_SUCCESS/AUTH_FAIL, COMMAND_STOP/RESUME, WEATHER_ALERT_ON/OFF, WEATHER_STOP/RESUME, UPDATE_LOCATION, REVOKE_KEY, errores, timeouts, etc.);
- d) parámetros del evento en formato JSON (cp_id, location, reason, etc.).

Este registro permite verificar el correcto funcionamiento del sistema y aporta trazabilidad para corrección y depuración.

9. Guía de despliegue

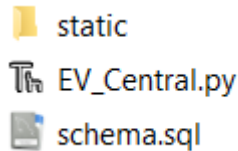
En primer lugar vamos a dejar clara la estructura de archivos y los distintos ordenadores que vamos a utilizar para la ejecución de este sistema distribuido. Vamos a utilizar 3 PCs, tal y como se refleja en la siguiente imagen que es parte de la memoria:



Instalación y preparación previa del sistema

PC 1

- Instalaremos y configuraremos kafka.
- Guardaremos la carpeta EV_Central que contiene los siguientes archivos:



EV_Central.py contiene toda la lógica de la central.

schema.sql contiene todas las instrucciones sql para generar la base de datos.

Y en static dentro contiene el index.html para poder general el front.

-Crearemos un .env y descargaremos todas las librerías y extensiones necesarias que detallaremos más adelante.

-Instalar python 3.13.1 mediante el siguiente enlace:

<https://www.python.org/downloads/windows/>

Acordarse de marcar add Python to path

-Instalar las siguientes dependencias (lo mas recomendado es dentro de un .venv)

pip install aiokafka

pip install fastapi uvicorn

pip install cryptography

pip install requests httpx

pip install pydantic

pip install websockets

-Instalar y configurar kafka, hay una guía muy útil en materiales docentes, acordarse de configurar correctamente las ips donde toca. Y de añadir KAFKA_HOME al path

PC 2

-Guardaremos las carpetas EV_CP_M, EV_CP_E, EV_W (dentro estan los correspondientes archivos .py)

-Crearemos un .env y descargamos todas las librerías y extensiones necesarias que detallaremos más adelante

-Copiamos el cert.pem que generamos en registry (explicado en PC3)

-Instalar python 3.13.1 mediante el siguiente enlace:

<https://www.python.org/downloads/windows/>

Acordarse de marcar add Python to path

-Instalar las siguientes dependencias (lo mas recomendado es dentro de un .venv)

pip install aiokafka

pip install cryptography

pip install requests httpx

PC 3

- Guardaremos las carpetas EV_Drivers y EV_Registry (dentro estan los correspondientes archivos .py) y en drivers el locations.txt
- Instalar python 3.13.1 mediante el siguiente enlace:
<https://www.python.org/downloads/windows/>
- Acordarse de marcar add Python to path
- Instalar las siguientes dependencias (lo mas recomendado es dentro de un .venv)
pip install aiokafka
pip install fastapi uvicorn
pip install requests httpx
pip install pydantic
- Crearemos el cert.pem y el key.pem

Para crear los certificados en primer lugar:

1. Modificaremos el archivo openssl-san.cnf (en la carpeta del Registry) con este contenido:

```
[req]
default_bits = 2048
prompt = no
default_md = sha256
distinguished_name = dn
x509_extensions = v3_req

[dn]
C = ES
ST = Madrid
L = Madrid
O = EVCharging
OU = Registry
CN = IP_PC3

[v3_req]
subjectAltName = @alt_names
keyUsage = digitalSignature, keyEncipherment
extendedKeyUsage = serverAuth

[alt_names]
IP.1 = IP_PC3
IP.2 = 127.0.0.1
DNS.1 = localhost
```

2. En power shell ejecutaremos la siguiente orden:
openssl req -x509 -newkey rsa:2048 -nodes ^
-keyout key.pem -out cert.pem -days 365 ^
-config openssl-san.cnf

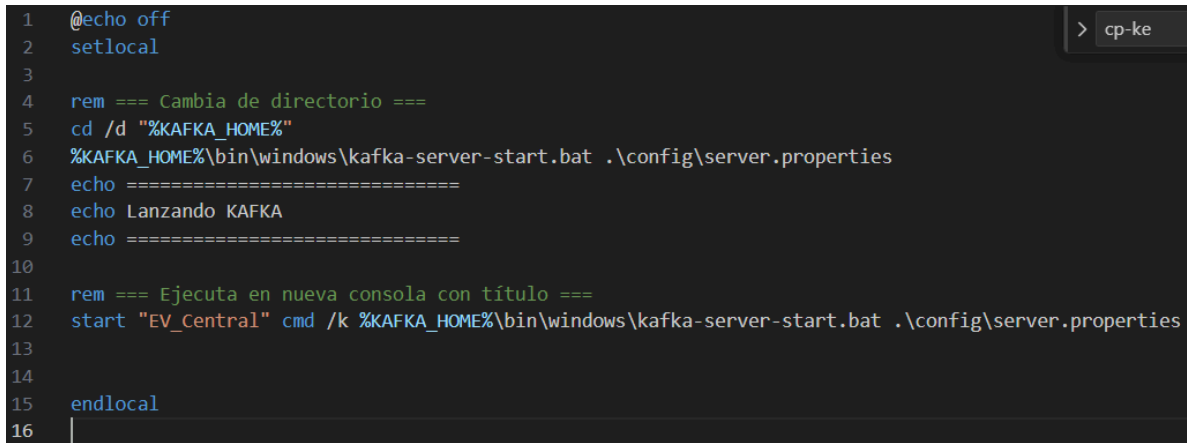
3. Se generarán un key.pem (no debemos moverlo de esta carpeta) y un cert.pem, este segundo deberemos copiarlo en la carpeta de EV_CP_Monitor (PC 2). Y ya estaría todo listo.

Comenzamos el despliegue

Tenemos una serie de .bat muy útiles que lanzan directamente los comandos necesarios, simplemente se deben modificar las correspondientes ips en cada archivo.

PC 1

Vamos a empezar lanzando kafka, podemos hacerlo ejecutando lanzakafka.bat:



```
1 @echo off
2 setlocal
3
4 rem === Cambia de directorio ===
5 cd /d "%KAFKA_HOME%"
6 %KAFKA_HOME%\bin\windows\kafka-server-start.bat .\config\server.properties
7 echo =====
8 echo Lanzando KAFKA
9 echo =====
10
11 rem === Ejecuta en nueva consola con título ===
12 start "EV_Central" cmd /k %KAFKA_HOME%\bin\windows\kafka-server-start.bat .\config\server.properties
13
14
15 endlocal
16 |
```

Si no, se puede hacer manual ejecutando:

```
cd %KAFKA_HOME%
%KAFKA_HOME%\bin\windows\kafka-server-start.bat .\config\server.properties
```

Ahora lanzamos central, lo podemos hacer mediante lanzacentral.bat

```

1  @echo off
2  setlocal
3
4  rem === CONFIGURACIÓN ===
5  set "RUTA_CENTRAL=D:\sd1\ds1\ds1\EVCharging\EV_Central"
6  set "SCRIPT=EV_Central.py"
7  set "HTTP_PORT=8081"
8  set "BROKER=127.0.0.1:9092"
9
10 rem === Cambia de directorio ===
11 cd /d "%RUTA_CENTRAL%"
12
13 echo =====
14 echo Lanzando EV_Central en %RUTA_CENTRAL%
15 echo Puerto HTTP: %HTTP_PORT%
16 echo Broker Kafka: %BROKER%
17 echo =====
18
19 rem === Ejecuta en nueva consola con título ===
20 start "EV_Central" cmd /k python "%SCRIPT%" %HTTP_PORT% %BROKER%
21
22 rem === Espera 1 segundo (opcional) ===
23 timeout /t 1 /nobreak >nul
24
25 endlocal
26

```

Hay que recordar de poner la ruta_central concreta que lleve al archivo donde tengamos EV_Central.py y en el broker poner la ip del ordenador donde se ha lanzado kafka que es el mismo ordenador.

Si no desde terminal:

python EV_Central.py <puerto_http> <ip_broker:puerto>

Abrimos el front en cualquier navegador:

<http://localhost:8081/static/index.html>

PC 2

Lanzamos engine con lanzaEngine.bat (esto lanzará 5 instancias de engine)

```

@echo off
setlocal

rem === CONFIG ===
set "BROKER=192.168.56.1:9092"
set "SCRIPT=D:\sd1\ds1\ds1\EVCharging\EV_CP_E\EV_CP_E.py"

for %%U in (6001 6002 6003 6004 6005) do (
    start "EV_CP_E %%U" cmd /k python "%SCRIPT%" "%BROKER%" %%U
    timeout /t 1 /nobreak >nul
)

endlocal

```

Debemos modificar el broker y la ruta al archivo
O directamente por terminal para cada instancia:
python EV_CP_E.py <broker_ip:puerto> <puerto_engine>

Lanzamos monitor con lanzaMonitor.bat (lanza 5 instancias de monitor)

```
@echo off
setlocal EnableDelayedExpansion

rem === CONFIGURACIÓN ===
set "BASE_PORT=6000"
set "BROKER=127.0.0.1:9000"
set "SCRIPT=D:\sd1\ds1\ds1\EVCharging\EV_CP_M\EV_CP_M.py"
set "REGISTRY=127.0.0.1:7070"

rem === LISTA DE PARKINGS (CADA UNO ENTRE COMILLAS) ===
set names="Alicante,ES" "Oslo,NO" "Barcelona,ES" "Madrid,ES" "Moscow,RU"

set prices[1]=0.45
set prices[2]=0.50
set prices[3]=0.49
set prices[4]=0.35
set prices[5]=0.43

set count=0
for %%A in (%names%) do (
    set /a count+=1
    set /a port=BASE_PORT+count

    call set "price=%%prices[!count!]%%"
    set "location=%%~A"

    echo =====
    echo Lanzando !location!
    echo -> Puerto: !port!
    echo -> Precio: !price!
    echo =====

    start "CP !count! !location!" cmd /k ^
    python "%SCRIPT%" 127.0.0.1:!port! %BROKER% %REGISTRY% !count! "!location!" !price!

    timeout /t 3 /nobreak >nul
)
```

Debemos modificar el broker, la ruta al archivo y la ip y puerto de registry

O por terminal:

python EV_CP_M.py <ip_engine:puerto> <ip_central:puerto> <ip_registry:puerto> <id_cp>
<location> <price>

Lanzamos weather con lanzaWeather.bat

```
@echo off
rem =====
rem Lanzador EV_W
rem =====

cd /d D:\sd1\ds1\ds1\EVCharging\EV_W

python EV_W.py http://127.0.0.1:8081 da93b07bb938a99ddf87e856393e53ce cities.txt

pause
```

O por terminal:

python EV_W.py http://127.0.0.1:8080 TU_API_KEY cities.txt

PC 3

Lanzamos registry con lanzaRegistry.bat

```
@echo off
echo Iniciando EV_Registry con Uvicorn...

cd /d D:\sd1\ds1\ds1\EVCharging\EV_Registry

python -m uvicorn EV_Registry:app ^
    --host 0.0.0.0 ^
    --port 7070 ^
    --ssl-keyfile "D:\sd1\ds1\ds1\EVCharging\EV_Registry\key.pem" ^
    --ssl-certfile "D:\sd1\ds1\ds1\EVCharging\EV_Registry\cert.pem"
pause
```

Debemos modificar las rutas del keyfile y y el ssl certfile.

O por terminal:

```
uvicorn EV_Registry:app --host 0.0.0.0 --port 7070 --ssl-keyfile key.pem --ssl-certfile cert.pem
```

Por último cuando queramos podemos lanzar los drivers con lanzaDrivers.bat (lanza 5)

```
@echo off
setlocal

rem === CONFIG ===
set "BROKER=192.168.56.1:9092"
set "SCRIPT=D:\sd1\ds1\ds1\EVCharging\EV_DRIVER\EV_Driver.py"
set "DEST=D:\sd1\ds1\ds1\EVCharging\EV_Driver\destinos.txt"

for %%U in (user1 user2 user3 user4 user5) do (
    start "EV_DRIVER %%U" cmd /k python "%SCRIPT%" %BROKER% %%U "%DEST%"
    timeout /t 4 /nobreak >nul
)

endlocal
```

Debemos modificar las rutas a los archivos y el broker.

O por terminal:

```
python EV_Driver.py <broker_ip:puerto> <id_cliente> [fichero_cp_ids]
```

Ejemplo del despliegue

Veremos el front de la siguiente forma nada mas desplegar todo menos los drivers:

⚡ EV Central - Panel de Monitorización ⚡

STOP ALL

RESUME ALL

ID	Ubicación	Precio (€/kWh)	Estado	kWh total	€ total	Driver	Última actualización	Acciones	Registrado	Clave
----	-----------	----------------	--------	-----------	---------	--------	----------------------	----------	------------	-------

Suministros activos

Inicio (fecha/hora)	Driver	CP	Session ID
---------------------	--------	----	------------

🌤 Weather (EV_W)

Ubicación	Temp (°C)	Alerta	Última actualización
Valencia,ES	10.54	OK (>=0°C)	2026-01-07T18:42:17
Alicante,ES	11.75	OK (>=0°C)	2026-01-07T18:42:17
Oslo,NO	-6.89	ALERTA (<0°C)	2026-01-07T18:42:17
Madrid,ES	5.52	OK (>=0°C)	2026-01-07T18:42:17
Moscow,RU	-3.83	ALERTA (<0°C)	2026-01-07T18:42:17

CPs deshabilitados por clima: —
CPs con stop pendiente: —

Últimos 5 mensajes

Fecha/Hora	Tipo	Detalle
2026-01-07T18:41:08	WEATHER_ALERT_OFF	{ "location": "Valencia,ES", "temp_c": 10.54 }
2026-01-07T18:41:09	WEATHER_ALERT_OFF	{ "location": "Alicante,ES", "temp_c": 11.75 }
2026-01-07T18:41:09	WEATHER_ALERT_ON	{ "location": "Oslo,NO", "temp_c": -6.89 }
2026-01-07T18:41:09	WEATHER_ALERT_OFF	{ "location": "Madrid,ES", "temp_c": 5.52 }
2026-01-07T18:41:09	WEATHER_ALERT_ON	{ "location": "Moscow,RU", "temp_c": -3.83 }

Estado de CENTRAL: OK

En monitor:

```
CP_1_Alicante,ES - python "D:\sd1\ds1\ds1\EVCharging\EV_CP_M\EV_CP_M.py" 127.0.0.1:6001 127.0.0.1:9000 127.0.0.1:7070 1 "Alicant...
EV_CP_M 1 | Engine:127.0.0.1:6001 | Central:127.0.0.1:9000 | Registry:127.0.0.1:7070

===== MENÚ CP_MONITOR =====
1) Dar de alta (Registry REST)
2) Dar de baja (Registry REST)
3) Autenticar (Central SOCKET)
4) Parar Heartbeats
5) Cambiar ubicación (UPDATE en Central)
0) Salir
=====

Opción:
```

Al dar de alta:

```
Opción: 1
Alta OK. Credential guardada en cp_1_credential.json

===== MENÚ CP_MONITOR =====
1) Dar de alta (Registry REST)
2) Dar de baja (Registry REST)
3) Autenticar (Central SOCKET)
4) Parar Heartbeats
5) Cambiar ubicación (UPDATE en Central)
0) Salir
=====

Opción:
```

STOP ALL

RESUME ALL

ID	Ubicación	Precio (€/kWh)	Estado	kWh total	€ total	Driver	Última actualización	Acciones	Registrado	Clave
1	Alicante,ES	0.45	DESCONECTADO	—	—	—	18:44:45	STOPRESUMEREVOKE KEY	ALTA	REVOCADA

Suministros activos

Al autenticar:

```
Opción: 3
[+] Secret key guardada en cp_1_secretkey.json
Autenticado en Central.
Engine ACK.
Heartbeats iniciados en background.

===== MENÚ CP_MONITOR =====
1) Dar de alta (Registry REST)
2) Dar de baja (Registry REST)
3) Autenticar (Central SOCKET)
4) Parar Heartbeats
5) Cambiar ubicación (UPDATE en Central)
0) Salir
=====

Opción: Heartbeat 1 (OK) -> Central: {'ok': True, 'action': 'HEARTBEAT_ACK', 'health': 'OK'}
Heartbeat 1 (OK) -> Central: {'ok': True, 'action': 'HEARTBEAT_ACK', 'health': 'OK'}
Heartbeat 1 (OK) -> Central: {'ok': True, 'action': 'HEARTBEAT_ACK', 'health': 'OK'}
Heartbeat 1 (OK) -> Central: {'ok': True, 'action': 'HEARTBEAT_ACK', 'health': 'OK'}
Heartbeat 1 (OK) -> Central: {'ok': True, 'action': 'HEARTBEAT_ACK', 'health': 'OK'}
```

ID	Ubicación	Precio (€/kWh)	Estado	kWh total	€ total	Driver	Última actualización	Acciones	Registrado	Clave
1	Alicante,ES	0.45	ACTIVADO	—	—	—	18:45:54	STOPRESUMEREVOKE KEY	ALTA	ACTIVA

GPS con stop pendiente: —

Últimos 5 mensajes

Fecha/Hora	Tipo	Detalle
2026-01-07T18:45:01	AUTH_SUCCESS	{"cp_id": "1", "location": "Alicante,ES", "price": 0.45}

Vamos a dar de alta y autenticar el resto de CP:

ID	Ubicación	Precio (€/kWh)	Estado	kWh total	€ total	Driver	Última actualización	Acciones	Registrado	Clave
1	Alicante,ES	0.45	ACTIVADO	—	—	—	18:47:40	STOPRESUMEREVOKE KEY	ALTA	ACTIVA
2	Oslo,NO	0.50	PARADO	—	—	—	18:47:40	STOPRESUMEREVOKE KEY	ALTA	ACTIVA
3	Barcelona,ES	0.49	ACTIVADO	—	—	—	18:47:40	STOPRESUMEREVOKE KEY	ALTA	ACTIVA
4	Madrid,ES	0.35	ACTIVADO	—	—	—	18:47:40	STOPRESUMEREVOKE KEY	ALTA	ACTIVA
5	Moscow,RU	0.43	PARADO	—	—	—	18:47:40	STOPRESUMEREVOKE KEY	ALTA	ACTIVA

Últimos 5 mensajes		
Fecha/Hora	Tipo	Detalle
2026-01-07T18:47:23	WEATHER_STOP	{"cp_id": "5", "location": "Moscow, RU"}
2026-01-07T18:47:23	COMMAND_SENT	{"source": "weather.alert_on", "action": "STOP", "cp_id": "5"}
2026-01-07T18:47:22	AUTH_SUCCESS	{"cp_id": "5", "location": "Moscow, RU", "price": 0.43}
2026-01-07T18:47:08	AUTH_SUCCESS	{"cp_id": "4", "location": "Madrid, ES", "price": 0.35}
2026-01-07T18:47:03	AUTH_SUCCESS	{"cp_id": "3", "location": "Barcelona, ES", "price": 0.49}

Weather (EV_W)			
Ubicación	Temp (°C)	Alerta	Última actualización
Valencia, ES	10.49	OK ($\geq 0^{\circ}\text{C}$)	2026-01-07T18:48:31
Alicante, ES	11.75	OK ($\geq 0^{\circ}\text{C}$)	2026-01-07T18:48:31
Oslo, NO	-6.89	ALERTA ($< 0^{\circ}\text{C}$)	2026-01-07T18:48:31
Madrid, ES	5.52	OK ($\geq 0^{\circ}\text{C}$)	2026-01-07T18:48:31
Moscow, RU	-3.83	ALERTA ($< 0^{\circ}\text{C}$)	2026-01-07T18:48:31

CPs deshabilitados por clima: 2, 5
CPs con stop pendiente: —

Como podemos ver están activos todos los cps que la temperatura lo permite.

Vamos a levantar 5 drivers, cada uno hará una pasada por los 5 CPs del 1 al 5.
En engine debemos aceptar el suministro:

```
===== MENÚ ENGINE =====
1) Activarse (health=OK / status=ACTIVADO)
2) Desactivar (KO) (health=KO / status=PARADO)
3) Suministrar a un driver manualmente
4) Aceptar suministro (si hay autorización pendiente)
5) Rechazar suministro (si hay autorización pendiente)
6) Finalizar suministro en curso (si hay)
=====

Escuchando conexiones del monitor en ('0.0.0.0', 6001)...
Recibido ID 1 y ubicación Alicante, ES
Estado actualizado: ACTIVADO
Autorización recibida para este CP. Usa 4) Aceptar o 5) Rechazar.
4
Aceptando suministro para driver user1 ...
```

```

===== MENÚ ENGINE =====
1) Activarse (health=OK / status=ACTIVADO)
2) Desactivar (KO) (health=KO / status=PARADO)
3) Suministrar a un driver manualmente
4) Aceptar suministro (si hay autorización pendiente)
5) Rechazar suministro (si hay autorización pendiente)
6) Finalizar suministro en curso (si hay)
=====

```

```

Estado actualizado: SUMINISTRANDO
Iniciando carga en 1 (Precio: 0.68 €/kWh)
1: 6.08 kW - 0.07 €
1: 6.35 kW - 0.14 €
1: 6.13 kW - 0.21 €
1: 6.42 kW - 0.28 €
1: 6.71 kW - 0.36 €
1: 7.41 kW - 0.44 €
1: 6.86 kW - 0.52 €

```

ID	Ubicación	Precio (€/kWh)	Estado	kWh total	€ total	Driver	Última actualización	Acciones	Registrado	Clave
1	Alicante,ES	0.45	SUMINISTRANDO	9.895	6.73	user1	18:55:02	STOP RESUME REVOKE KEY	ALTA	ACTIVA

Suministros activos

Inicio (fecha/hora)	Driver	CP	Session ID
2026-01-07 17:50:17	user1	1	1
2026-01-07 17:50:29	user2	3	2
2026-01-07 17:50:37	user3	4	3

Vamos a aceptar los tres suministros:

ID	Ubicación	Precio (€/kWh)	Estado	kWh total	€ total	Driver	Última actualización	Acciones	Registrado	Clave
1	Alicante,ES	0.45	SUMINISTRANDO	16.678	11.34	user1	18:56:04	STOP RESUME REVOKE KEY	ALTA	ACTIVA
2	Oslo,NO	0.50	PARADO	—	—	—	18:56:03	STOP RESUME REVOKE KEY	ALTA	ACTIVA
3	Barcelona,ES	0.49	SUMINISTRANDO	1.161	0.65	user2	18:56:03	STOP RESUME REVOKE KEY	ALTA	ACTIVA
4	Madrid,ES	0.35	SUMINISTRANDO	0.666	0.28	user3	18:56:04	STOP RESUME REVOKE KEY	ALTA	ACTIVA
5	Moscow,RU	0.43	PARADO	—	—	—	18:56:03	STOP RESUME REVOKE KEY	ALTA	ACTIVA

Como podemos observar el CP 2 y 3 estan parados debido a la temperatura:

```

C:\> EV_CP_E 6002 - python "D:\sd1\ds1\ds1\EVCharging\EV_CP_E\EV_CP_E.py" "192.168.56.1:9092" 6002
EV_CP_E conectado a Kafka 192.168.56.1:9092
Puerto del Engine para monitor: 6002
Esperando mensajes de CENTRAL...

===== MENÚ ENGINE =====
1) Activarse (health=OK / status=ACTIVADO)
2) Desactivar (KO) (health=KO / status=PARADO)
3) Suministrar a un driver manualmente
4) Aceptar suministro (si hay autorización pendiente)
5) Rechazar suministro (si hay autorización pendiente)
6) Finalizar suministro en curso (si hay)
=====

Escuchando conexiones del monitor en ('0.0.0.0', 6002)...
Recibido ID 2 y ubicación Oslo,NO
Estado actualizado: ACTIVADO
Estado actualizado: PARADO
(Aviso) STOP recibido en 2

```

Desde monitor podemos cambiar su ubicacion:

Con la opción 5 y poniendo en el formado correcto la ubicacion:

Ciudad,INICIALES_PAIS

Debemos asegurarnos que en weather estamos controlando esa ubicación se puede añadir pulsando 4 e introduciendo la ubicacion

```

OK añadido: Elche,ES

===== EV_W MENU =====
1) Listar ciudades
2) Añadir ciudad
3) Eliminar ciudad
4) Ver estados (temp/alert)
q) Salir
=====

```

5	Elche,ES	0.43	ACTIVADO	—	—	—	19:00:56	STOP	RESUME	REVOKE KEY	ALTA	ACTIVA
---	----------	------	----------	---	---	---	----------	------	--------	------------	------	--------

Podemos finalizar un suministro con el 6 en Engine:

```

6
Solicitud de finalización de carga enviada.

===== MENÚ ENGINE =====
1) Activarse (health=OK / status=ACTIVADO)
2) Desactivar (KO) (health=KO / status=PARADO)
3) Suministrar a un driver manualmente
4) Aceptar suministro (si hay autorización pendiente)
5) Rechazar suministro (si hay autorización pendiente)
6) Finalizar suministro en curso (si hay)
=====

Estado actualizado: ACTIVADO
Carga finalizada. reason=FINALIZADO_CORRECTAMENTE_POR_ENGINE, final_status=ACTIVADO

===== MENÚ ENGINE =====
1) Activarse (health=OK / status=ACTIVADO)
2) Desactivar (KO) (health=KO / status=PARADO)
3) Suministrar a un driver manualmente
4) Aceptar suministro (si hay autorización pendiente)
5) Rechazar suministro (si hay autorización pendiente)
6) Finalizar suministro en curso (si hay)
=====

```

A driver se le manda el ticket y seguirá la lista de peticiones de suministros buscando uno disponible:

```

1 → 7.29 kW | 53.0810 kWh | 36.18 €
2 → 7.3 kW | 53.2030 kWh | 36.18 €
Update: FINISHED – Servicio finalizado en 1

===== TICKET DE RECARGA =====
CP: 1
Localización: Alicante,ES
Precio/kWh: 0.45 €
Energía: 53.2030 kWh
Importe: 36.18 €
Estado: FINALIZADO_CORRECTAMENTE_POR_ENGINE
=====

Solicitud enviada → CP=2 | request_id=e00b7cb9-f08a-42c1-8e96-f851f7f4f6c1
Update: DENIED – CP 2 no disponible (estado PARADO)
Solicitud enviada → CP=3 | request_id=d6eea6a8-cbf9-4c75-b75d-e9f98e0cf3a8
Update: DENIED – CP 3 ocupado: hay una sesión en curso
Solicitud enviada → CP=4 | request_id=10cd9855-c1e1-4596-a853-274072062491
Update: DENIED – CP 4 ocupado: hay una sesión en curso
Solicitud enviada → CP=5 | request_id=f88fcf63-1962-4771-9b07-68b6e891ee1f
Update: AUTHORIZED – Autorizado en 5

```

Últimos 5 mensajes		
Fecha/Hora	Tipo	Detalle
2026-01-07T19:01:48	SUMINISTRO_SOLICITADO	{"cp_id":"5","driver_id":"user1","session_id":4}
2026-01-07T19:01:32	SUMINISTRO_FINALIZADO	{"cp_id":"1","reason":"FINALIZADO_CORRECTAMENTE_POR_ENGINE","driver_id":"user1","kwh":53.203,"amount_eur":36.18}

El engine vuelve a estado activado.

Si pulsamos stop all:

STOP ALLRESUME ALL										
ID	Ubicación	Precio (€/kWh)	Estado	kWh total	€ total	Driver	Última actualización	Acciones	Registrado	Clave
1	Alicante,ES	0.45	PARADO	—	—	—	19:05:36	STOPRESUMEREVOKE KEY	ALTA	ACTIVA
2	Oslo,NO	0.50	PARADO	—	—	—	19:05:36	STOPRESUMEREVOKE KEY	ALTA	ACTIVA
3	Barcelona,ES	0.49	PARADO	—	—	—	19:05:36	STOPRESUMEREVOKE KEY	ALTA	ACTIVA
4	Madrid,ES	0.35	PARADO	—	—	—	19:05:36	STOPRESUMEREVOKE KEY	ALTA	ACTIVA
5	Elche,ES	0.43	PARADO	—	—	—	19:05:36	STOPRESUMEREVOKE KEY	ALTA	ACTIVA

Fecha/Hora	Tipo	Detalle
2026-01-07T19:05:33	SUMINISTRO_FINALIZADO	{"cp_id":"4","reason":"FINALIZADO_POR_STOP_DE_CENTRAL","driver_id":"user3","kwh":63.916,"amount_eur":26.84}
2026-01-07T19:05:33	SUMINISTRO_FINALIZADO	{"cp_id":"3","reason":"FINALIZADO_POR_STOP_DE_CENTRAL","driver_id":"user2","kwh":64.569,"amount_eur":36.16}

```
4 → 6.9 kW | 63.9160 kWh | 26.84 €
Update: FINISHED – Servicio finalizado en 4

===== TICKET DE RECARGA =====
CP: 4
Localización: Madrid,ES
Precio/kWh: 0.35 €
Energía: 63.9160 kWh
Importe: 26.84 €
Estado: FINALIZADO_POR_STOP_DE_CENTRAL
=====
```

Si pulsamos resume all todo vuelve a la normalidad:

STOP ALLRESUME ALL										
ID	Ubicación	Precio (€/kWh)	Estado	kWh total	€ total	Driver	Última actualización	Acciones	Registrado	Clave
1	Alicante,ES	0.45	ACTIVADO	—	—	—	19:07:06	STOPRESUMEREVOKE KEY	ALTA	ACTIVA
2	Oslo,NO	0.50	PARADO	—	—	—	19:07:06	STOPRESUMEREVOKE KEY	ALTA	ACTIVA
3	Barcelona,ES	0.49	ACTIVADO	—	—	—	19:07:06	STOPRESUMEREVOKE KEY	ALTA	ACTIVA
4	Madrid,ES	0.35	ACTIVADO	—	—	—	19:07:06	STOPRESUMEREVOKE KEY	ALTA	ACTIVA
5	Elche,ES	0.43	ACTIVADO	—	—	—	19:07:06	STOPRESUMEREVOKE KEY	ALTA	ACTIVA

Si revocamos la key:

ID	Ubicación	Precio (€/kWh)	Estado	kWh total	€ total	Driver	Última actualización	Acciones	Registrado	Clave
1	Alicante,ES	0.45	DESCONECTADO	—	—	—	19:07:44	STOP RESUME REVOKE KEY	ALTA	REVOCADA

```
Heartbeat -> Central error: Respuesta no-JSON de central: DENIED:BAD_SECURE_MESSAGE
Heartbeat -> Central error: Respuesta no-JSON de central: DENIED:BAD_SECURE_MESSAGE
Heartbeat -> Central error: Respuesta no-JSON de central: DENIED:BAD_SECURE_MESSAGE
Heartbeat -> Central error: Respuesta no-JSON de central: DENIED:BAD_SECURE_MESSAGE
```

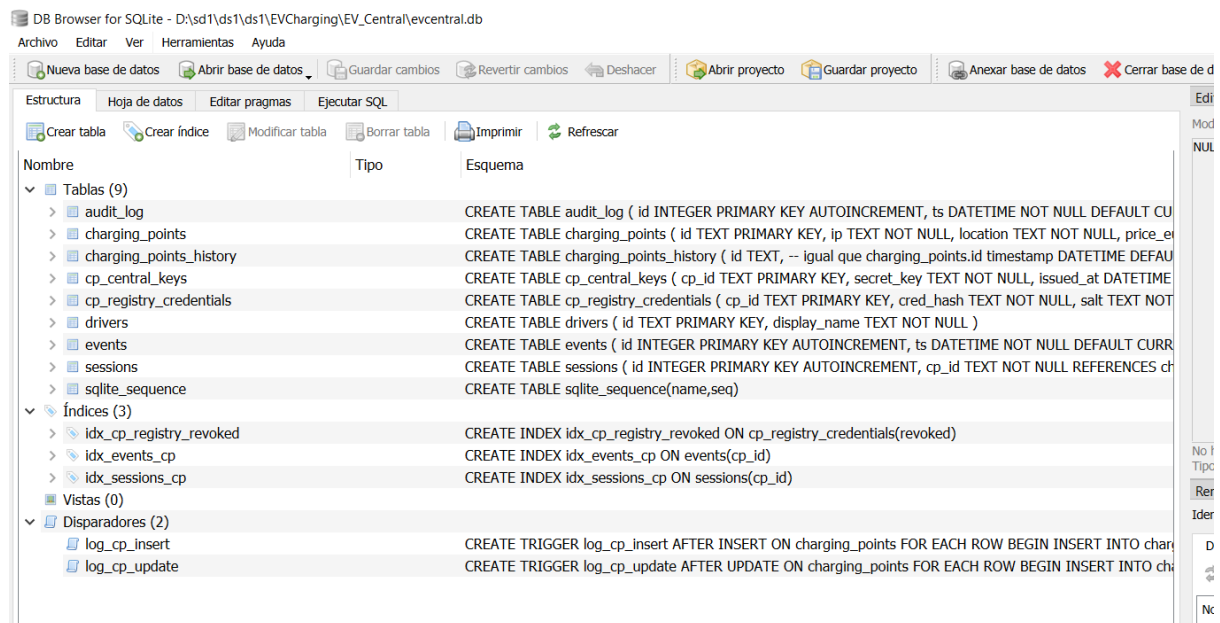
Monitor sigue mandando heartbeats pero al revocar la key central no los acepta
Podemos volver a autenticarnos con la opción 3:

ID	Ubicación	Precio (€/kWh)	Estado	kWh total	€ total	Driver	Última actualización	Acciones	Registrado	Clave
1	Alicante,ES	0.45	ACTIVADO	—	—	—	19:10:24	STOP RESUME REVOKE KEY	ALTA	ACTIVA

10. Aplicaciones extra utilizadas

DB Browser for SQLite

Hemos utilizado DB Browser for SQLite para hacer selects en tiempo de ejecución y comprobar que se añadían correctamente los datos en la base de datos.



Github

Hemos creado un repositorio de github para desarrollar la aplicación con 4 ramas

master para las versiones funcionales

main para las versiones en desarrollo

quevedo para el desarrollo intermedio antes de subir a main por quevedo

pablo para el desarrollo intermedio antes de subir a main por pablo

WireShark

Hemos utilizado wireshark para comprobar si realmente se estaban cifrando las tramas que se estaban enviando los distintos módulos y también un poco por curiosidad del estado en el que estaban las tramas.